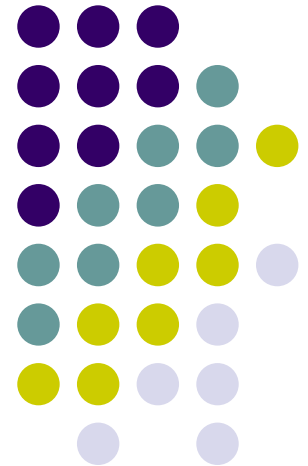
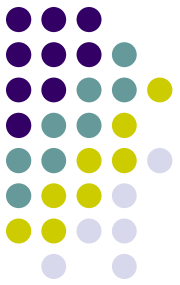


Iterator



Diner and Pancake House Merge

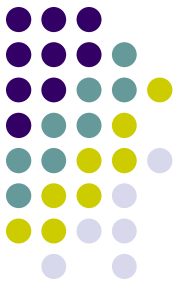


They want to use my Pancake House menu as the breakfast menu and the Diner's menu as the lunch menu. We've agreed on an implementation for the menu items...

that depends on them.

en

Menultem – They agree the same Menultem implementation



```
public class MenuItem {
    String name;
    String description;
    boolean vegetarian;
    double price;

    public MenuItem(String name,
                    String description,
                    boolean vegetarian,
                    double price)
    {
        this.name = name;
        this.description = description;
        this.vegetarian = vegetarian;
        this.price = price;
    }

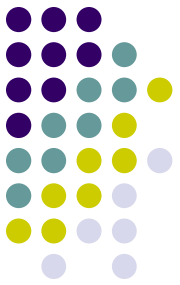
    public String getName() {
        return name;
    }

    public String getDescription() {
        return description;
    }

    public double getPrice() {
        return price;
    }

    public boolean isVegetarian() {
        return vegetarian;
    }
}
```

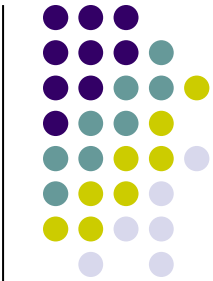
4/8/2025



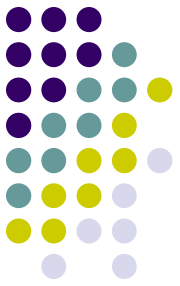


em has a name, a
description, whether or not it's a
vegetarian item, and the price





The Java-Enabled Waitress Specification



Java-Enabled Waitress: code-name "Alice"

`printMenu()`
- prints every item on the menu

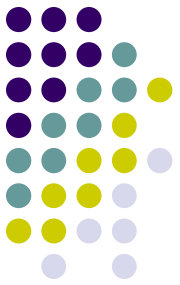
`printBreakfastMenu()`
- prints just breakfast items

`printLunchMenu()`
- prints just lunch items

`printVegetarianMenu()`
- prints all vegetarian menu items

`isItemVegetarian(name)`
- given the name of an item, returns true
if the item is vegetarian, otherwise,
returns false

PrintMenu



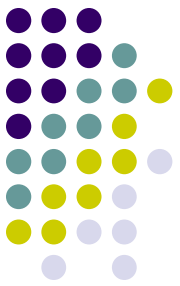
- 1 To print all the items on each menu, you'll need to call the `getMenuItem()` method on the `PancakeHouseMenu` and the `DinerMenu` to retrieve their respective menu items. Note that each returns a different type:

```
PancakeHouseMenu pancakeHouseMenu = new PancakeHouseMenu();  
ArrayList breakfastItems = pancakeHouseMenu.getMenuItems();
```

```
DinerMenu dinerMenu = new DinerMenu();  
MenuItem[] lunchItems = dinerMenu.getMenuItems();
```

The method looks the same, but the calls are returning different types.

The implementation is showing through, breakfast items are in an `ArrayList`, lunch items are in an `Array`.



- 2 Now, to print out the items from the PancakeHouseMenu, we'll loop through the items on the breakfastItems ArrayList. And to print out the Diner items we'll loop through the Array.

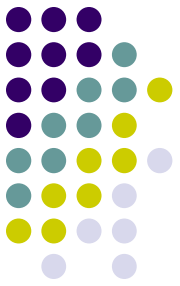
```
for (int i = 0; i < breakfastItems.size(); i++) {  
    MenuItem menuItem = (MenuItem)breakfastItems.get(i);  
    System.out.print(menuItem.getName() + " ");  
    System.out.println(menuItem.getPrice() + " ");  
    System.out.println(menuItem.getDescription());  
}  
  
for (int i = 0; i < lunchItems.length; i++) {  
    MenuItem menuItem = lunchItems[i];  
    System.out.print(menuItem.getName() + " ");  
    System.out.println(menuItem.getPrice() + " ");  
    System.out.println(menuItem.getDescription());  
}
```

← Now, we have to implement two different loops to step through the two implementations of the menu items...

← ...one loop for the ArrayList...

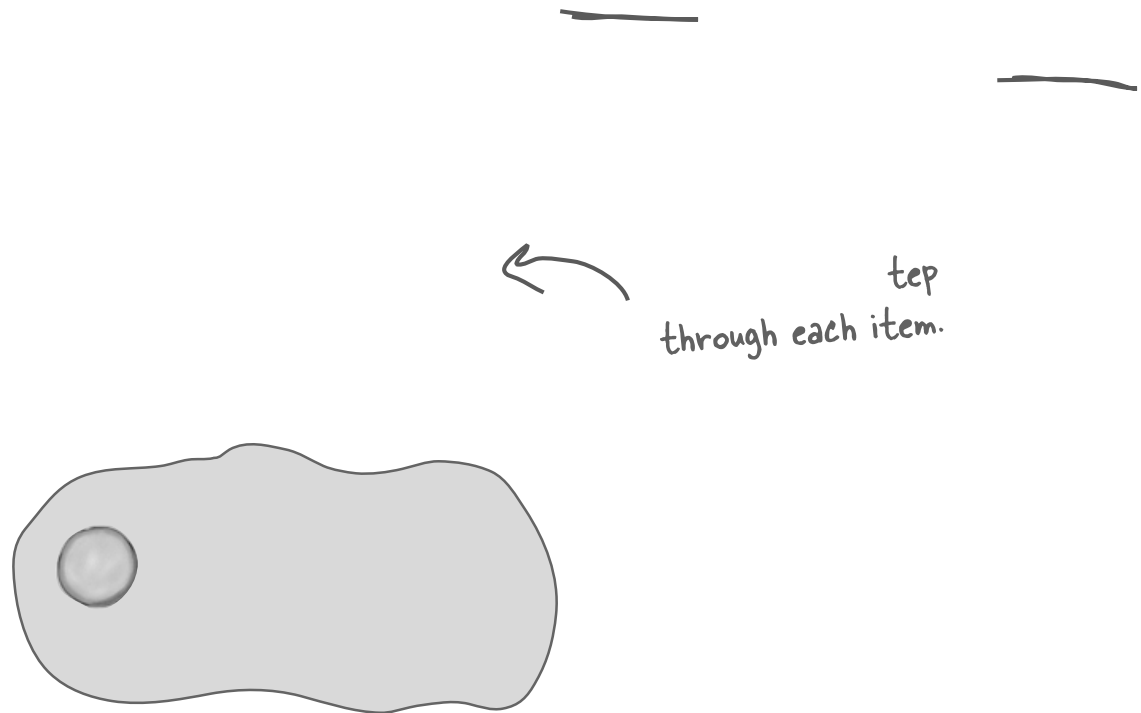
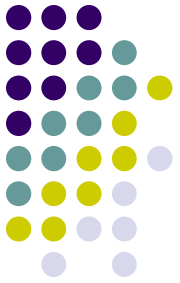
← and another for the Array.

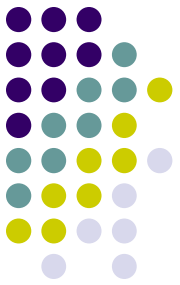
3



- Implementing every other method in the Waitress is going to be a variation of this theme. We're always going to need to get both menus and use two loops to iterate through their items. If another restaurant with a different implementation is acquired then we'll have three loops.

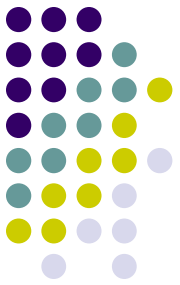
Encapsulate the Iteration?





We use the array
subscripts to step
through items.

A hand-drawn arrow pointing to the right, starting from the text and curving upwards at the end.



- 3 Now what if we create an object, let's call it an Iterator, that encapsulates the way we iterate through a collection of objects? Let's try this on the ArrayList

```
Iterator iterator = breakfastMenu.createIterator();
```

```
while (iterator.hasNext()) {  
    MenuItem menuItem = (MenuItem)iterator.next();  
}
```

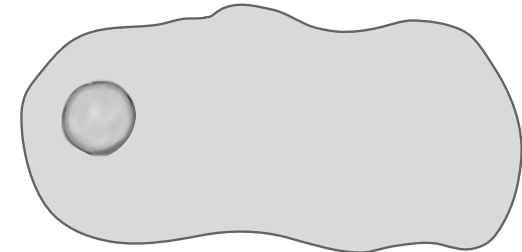
next()

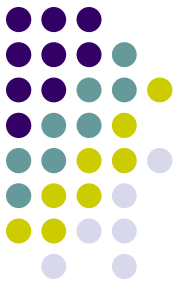
get(2)

get(3)

get(1)

get(0)



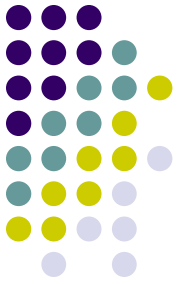


the
breakfastMenu
code.

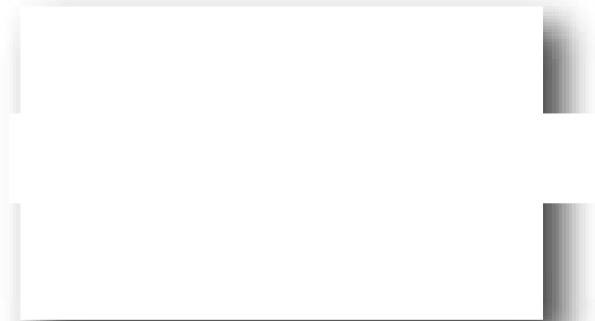


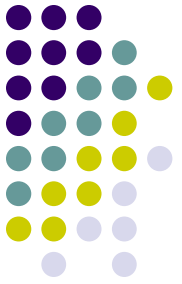
next()

Meet the Iterator Pattern

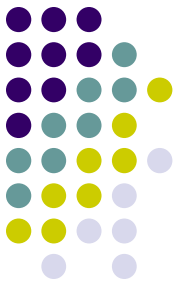


One possible interface





Adding an Iterator to DinerMenu



To add an Iterator to the DinerMenu we first need to define the Iterator Interface:

```
public interface Iterator {  
    boolean hasNext();  
    Object next();  
}
```

Here's our two methods:

The hasNext() method returns a boolean indicating whether or not there are more elements to iterate over...

...and the next() method returns the next element.

```
public class DinerMenuIterator implements Iterator {  
    MenuItem[] items;  
    int position = 0;
```

position maintains the current position of the iteration over the array.

```
    public DinerMenuIterator(MenuItem[] items) {  
        this.items = items;  
    }
```

The constructor takes the array of menu items we are going to iterate over.

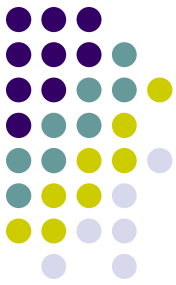
```
    public Object next() {  
        MenuItem menuItem = items[position];  
        position = position + 1;  
        return menuItem;  
    }
```

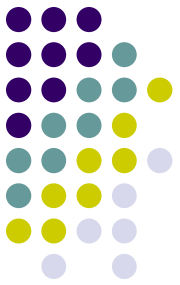
The next() method returns the next item in the array and increments the position.

```
    public boolean hasNext() {  
        if (position >= items.length || items[position] == null) {  
            return false;  
        } else {  
            return true;  
        }  
    }
```

The hasNext() method checks to see if we've seen all the elements of the array and returns true if there are more to iterate through.

Because the diner chef went ahead and allocated a max sized array, we need to check not only if we are at the end of the array, but also if the next item is null, which indicates there are no more items.





```
public class DinerMenu {
    static final int MAX_ITEMS = 6;
    int numberOfItems = 0;
    MenuItem[] menuItems;

    // constructor here

    // addItem here

public MenuItem[] getMenuItems() {
    return menuItems;
}

    public Iterator createIterator() {
        return new DinerMenuIterator(menuItems);
    }

    // other menu methods here
}
```

We're not going to need the getMenuItems() method anymore and in fact, we don't want it because it exposes our internal implementation!

Here's the createIterator() method. It creates a DinerMenuIterator from the menuItems array and returns it to the client.

We're returning the Iterator interface. The client doesn't need to know how the menuItems are maintained in the DinerMenu, nor does it need to know how the DinerMenuIterator is implemented. It just needs to use the iterators to step through the items in the menu.

```
public class Waitress {
    PancakeHouseMenu pancakeHouseMenu;
    DinerMenu dinerMenu;
```

In the constructor the Waitress takes the two menus.

```
public Waitress(PancakeHouseMenu pancakeHouseMenu, DinerMenu dinerMenu) {
    this.pancakeHouseMenu = pancakeHouseMenu;
    this.dinerMenu = dinerMenu;
}
```

The printMenu() method now creates two iterators, one for each menu.

```
public void printMenu() {
    Iterator pancakeIterator = pancakeHouseMenu.createIterator();
    Iterator dinerIterator = dinerMenu.createIterator();
    System.out.println("MENU\n----\nBREAKFAST");
    printMenu(pancakeIterator);
    System.out.println("\nLUNCH");
    printMenu(dinerIterator);
}
```

And then calls the overloaded printMenu() with each iterator.

```
private void printMenu(Iterator iterator) {
    while (iterator.hasNext()) {
        MenuItem menuItem = (MenuItem)iterator.next();
        System.out.print(menuItem.getName() + ", ");
        System.out.print(menuItem.getPrice() + " -- ");
        System.out.println(menuItem.getDescription());
    }
}
```

Test if there are any more items.

Get the next item.

The overloaded printMenu() method uses the Iterator to step through the menu items and print them.

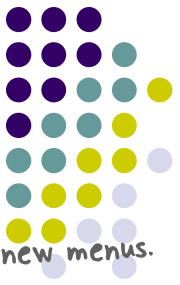
```
// other methods here
```

Note that we're down to one loop.

Use the item to get name, price and description and print them.



Testing



First we create the new menus.

```
public class MenuTestDrive {  
    public static void main(String args[]) {  
        PancakeHouseMenu pancakeHouseMenu = new PancakeHouseMenu();  
        DinerMenu dinerMenu = new DinerMenu();
```

```
        Waitress waitress = new Waitress(pancakeHouseMenu, dinerMenu);  
        waitress.printMenu();  
    }  
}
```

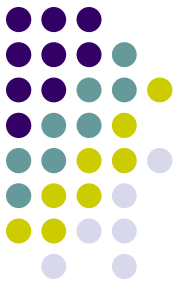
Then we create a Waitress and pass her the menus.

Then we print them.

```
File Edit Window Help GreenEggs&Ham  
% java DinerMenuTestDrive  
MENU  
----  
BREAKFAST  
K&B's Pancake Breakfast, 2.99 -- Pancakes with scrambled eggs, and toast  
Regular Pancake Breakfast, 2.99 -- Pancakes with fried eggs, sausage  
Blueberry Pancakes, 3.49 -- Pancakes made with fresh blueberries  
Waffles, 3.59 -- Waffles, with your choice of blueberries or strawberries  
  
LUNCH  
Vegetarian BLT, 2.99 -- (Fakin') Bacon with lettuce & tomato on whole wheat  
BLT, 2.99 -- Bacon with lettuce & tomato on whole wheat  
Soup of the day, 3.29 -- Soup of the day, with a side of potato salad  
Hotdog, 3.05 -- A hot dog, with saurkraut, relish, onions, topped with cheese  
Steamed Veggies and Brown Rice, 3.99 -- Steamed vegetables over brown rice  
Pasta, 3.89 -- Spaghetti with Marinara Sauce, and a slice of sourdough bread  
  
%
```

First we iterate through the pancake menu.

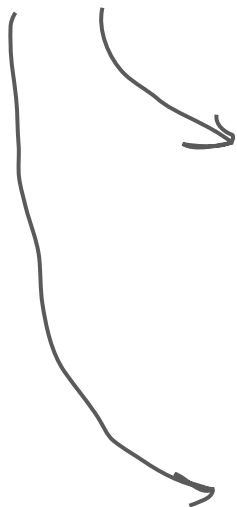
And then the lunch menu, all with the same iteration code.



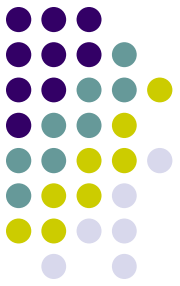
nterface, which
means the Waitress is still bound to
two concrete Menu classes. We'd
better fix that.

TM

or to do her iterating.



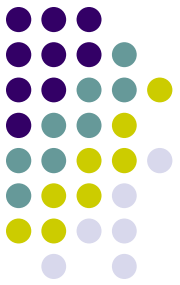
Cleaning things with `java.util.Iterator`



- PancakeHouseMenu:

```
public Iterator createIterator() {  
    return menuItems.iterator();  
}
```

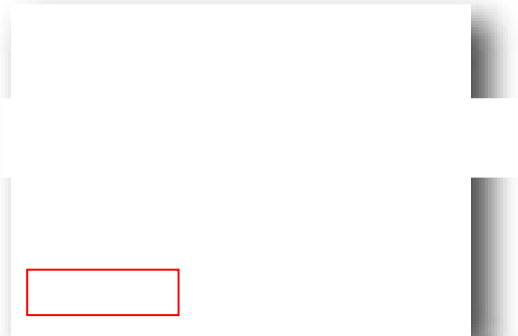
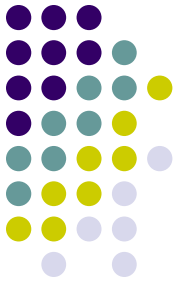
← Instead of creating our own iterator now, we just call the `iterator()` method on the `menuItems ArrayList`.



Java Iterator

- Collection is the root interface of all Java aggregate classes.
- In the JDK class library, the `iterator()` method of `Collection` returns an object of type `java.util.Iterator`, and the `listIterator()` method of its subinterface `java.util.List` returns an object of type `java.util.ListIterator`, and `ListIterator` is Subclass of `Iterator`. They constitute the Java language's support for the iterator pattern, and the `java.util.Iterator` interface of the Java language is the application of the iterator pattern.

Java Iterator



There is no `iterator()` for an Array.

```
import java.util.Iterator;
```

```
public class DinerMenuIterator implements Iterator {
```

```
    MenuItem[] list;
```

```
    int position = 0;
```

```
    public DinerMenuIterator(MenuItem[] list) {
```

```
        this.list = list;
```

```
    }
```

```
    public Object next() {
```

```
        //implementation here
```

```
    }
```

```
    public boolean hasNext() {
```

```
        //implementation here
```

```
    }
```

```
    public void remove() {
```

```
        if (position <= 0) {
```

```
            throw new IllegalStateException
```

```
                ("You can't remove an item until you've done at least one next()");
```

```
        }
```

```
        if (list[position-1] != null) {
```

```
            for (int i = position-1; i < (list.length-1); i++) {
```

```
                list[i] = list[i+1];
```

```
            }
```

```
            list[list.length-1] = null;
```

```
        }
```

```
    }
```

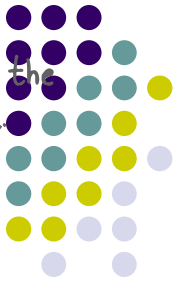
```
}
```

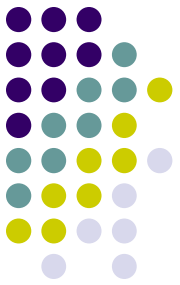
4/8/2025

First we import java.util.Iterator, the interface we're going to implement.

None of our current implementation changes...

...but we do need to implement remove(). Here, because the chef is using a fixed sized Array, we just shift all the elements up one when remove() is called.





```
import java.util.Iterator;
```

Now the Waitress uses the java.util.Iterator as well.

```
public class Waitress {  
    Menu pancakeHouseMenu;  
    Menu dinerMenu;
```

```
    public Waitress(Menu pancakeHouseMenu, Menu dinerMenu) {  
        this.pancakeHouseMenu = pancakeHouseMenu;  
        this.dinerMenu = dinerMenu;  
    }
```

We need to replace the concrete Menu classes with the Menu Interface.

Public interface Menu{

public Iterator createIterator();

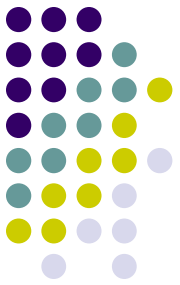
```
    public void printMenu() {  
        Iterator pancakeIterator = pancakeHouseMenu.createIterator();  
        Iterator dinerIterator = dinerMenu.createIterator();  
        System.out.println("MENU\n----\nBREAKFAST");  
        printMenu(pancakeIterator);  
        System.out.println("\nLUNCH");  
        printMenu(dinerIterator);  
    }
```

Nothing changes here.

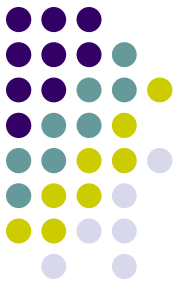
```
    private void printMenu(Iterator iterator) {  
        while (iterator.hasNext()) {  
            MenuItem menuItem = (MenuItem) iterator.next();  
            System.out.print(menuItem.getName() + ", ");  
            System.out.print(menuItem.getPrice() + " -- ");  
            System.out.println(menuItem.getDescription());  
        }  
    }
```

```
    // other methods here
```

```
}
```



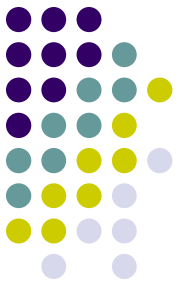
- The PancakeHouseMenu and DinerMenu classes implement an interface, Menu. Waitress can refer to each menu object using the interface rather than the concrete class. So, we're reducing the dependency between the Waitress and the concrete classes by
“programming to an interface, not an implementation.”
- The new Menu interface has one method, createIterator(), that is implemented by PancakeHouseMenu and DinerMenu. Each menu class assumes the responsibility of creating a concrete Iterator that is appropriate for its internal implementation of the menu items.



Waitress
printMenu()



n't need this anymore.

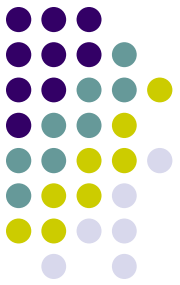


Iterator Pattern defined

- The Iterator Pattern provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation.
- It also places the task of traversal on the iterator object, not on the aggregate, which simplifies the aggregate interface and implementation, and places the responsibility where it should be.

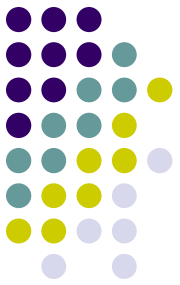


ble for managing
the current position of
the iteration.



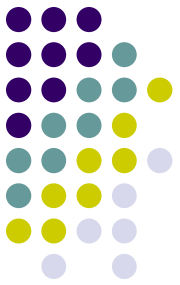
Single Responsibility

- Design Principle
 - A class should have only one reason to change.
- Every responsibility of a class is an area of potential change. More than one responsibility means more than one area of change.
- This principles guides us to keep each class to a single responsibility.
- The only way to succeed is to be diligent in examining your designs and to watch out for signals that a class is changing in more than one way as your system grows.



Iterators and Collections

- We've been using a couple of classes that are part of the Java Collections Framework. This "framework" is just a set of classes and interfaces, including ArrayList, which we've been using, and many others like Vector, LinkedList, Stack, and PriorityQueue. Each of these classes implements the `java.util.Collection` interface, which contains a bunch of useful methods for manipulating groups of objects.



Iterators and Collections

<<interface>> Collection
<code>add()</code>
<code>addAll()</code>
<code>clear()</code>
<code>contains()</code>
<code>containsAll()</code>
<code>equals()</code>
<code>hashCode()</code>
<code>isEmpty()</code>
<code>iterator()</code>
<code>remove()</code>
<code>removeAll()</code>
<code>retainAll()</code>
<code>size()</code>
<code>toArray()</code>

As you can see, there's all kinds of good stuff here. You can add and remove elements from your collection without even knowing how it's implemented.

Here's our old friend, the `iterator()` method. With this method, you can get an `Iterator` for any class that implements the `Collection` interface.

Other handy methods include `size()`, to get the number of elements, and `toArray()` to turn your collection into an array.